



UNIVERSIDAD
SAN SEBASTIAN

USS-ICIFH001 / INTELIGENCIA ARTIFICIAL / CLASE 10

Lab 05 – El gato con minimax

Cristhian Aguilera

2026-08-17

Nivel 1

Turno de X
X maximiza

Nivel 2

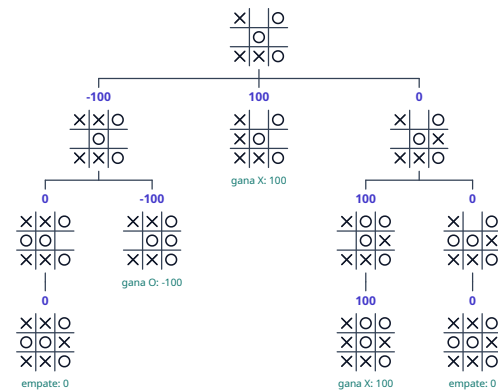
Turno de O
O minimiza

Nivel 3

Turno de X
X maximiza

Nivel 4

Estados finales
utilidad para X



Contenidos

1. Objetivos del Laboratorio

2. Tarea Lab 05

Objetivos del Laboratorio

Al finalizar este laboratorio, los estudiantes podrán:

Traducir la definición recursiva de minimax a código, sobre las funciones que definen el juego.

Construir un jugador que elige la acción óptima: maximiza con MAX y minimiza con MIN.

Medir el costo de la búsqueda en nodos explorados y tiempo por jugada.

Cortar (*opcional*) la búsqueda por profundidad y reemplazar la utilidad por una heurística, para graduar la fuerza de la máquina.

01

SECCIÓN

Tarea Lab 05

- 1.1 Condiciones de la tarea
- 1.2 Bonus - niveles de dificultad
- 1.3 Rúbrica y entrega

Tarea: que la máquina no pierda nunca

🕒 90 min · Individual

Parte del proyecto `tictactoe_skeleton.tar` de Blackboard: tablero, dibujo y turnos ya están hechos.

`minimax.py` es el único archivo que se edita.

1. `minimax(state, stats)`: la recursión de la clase.
2. El contador de nodos: +1 en cada llamada.
3. `choose_move`: máximo con CROSS, mínimo con CIRCLE.
4. Usar `board.py`, no reescribir las reglas.
5. Una máquina que **no pierda nunca**, con X y con O.

```
1 lab05-tictactoe/  
2   └─ main.py  
3   └─ tictactoe_lab/  
4       └─ board.py    # las reglas  
5       └─ minimax.py  # ← tú  
6       └─ renderer.py  
7       └─ game.py
```

TecLa	Acción
click	poner tu ficha
R / ESC	nueva partida / salir
1 2 3	niveles, solo con el bonus

Bonus: niveles de dificultad

Con el árbol completo la máquina es imbatible y no hay nada que graduar. Los niveles `easy` y `medium` salen de **cortar la búsqueda**:

$$h\text{-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{si } d = 0 \\ \max_a \dots & \text{si player}(s) = \text{MAX} \\ \min_a \dots & \text{si player}(s) = \text{MIN} \end{cases}$$

Hay que **agregar** dos funciones a `minimax.py`: `evaluate(state)`, la heurística, y `h_minimax(state, depth, stats)`, que la usa al llegar al corte.

Nivel	Profundidad
<code>easy</code>	1
<code>medium</code>	3
<code>hard</code>	sin corte

Rúbrica y entrega

Cada condición se cumple o no se cumple, sin puntaje parcial, revisado ejecutando tu juego.

#	Condición	Puntos
1	<code>minimax</code> es la recursión de la clase: terminal devuelve <code>utility</code> , MAX toma el máximo, MIN el mínimo, y suma 1 nodo en cada llamada.	2
2	<code>choose_move</code> prueba todas las acciones legales, maximiza con CROSS y minimiza con CIRCLE, y devuelve <code>(move, stats)</code> .	2
3	Las reglas del gato no se reimplementan dentro de <code>minimax.py</code> : se usan las funciones de <code>board.py</code> .	1
4	<code>minimax</code> sobre el tablero vacío vale 0 y en las partidas de prueba la máquina empata o gana, nunca pierde , con X y con O.	2
★	Bonus: <code>evaluate</code> y <code>h_minimax</code> agregados, <code>easy</code> y <code>medium</code> juegan y se les puede ganar.	+1

Total: 7 puntos. El bonus recupera un punto perdido, la nota no pasa de 7.

- **Entrega:** el proyecto `uv` comprimido (`lab05-apellido.tar`).
- Debe correr con `uv run python main.py` y se sube a blackboard.



UNIVERSIDAD
SAN SEBASTIAN

USS-ICIFH001 • CLASE 10

¡Fin del Lab 5
